

PROJECT REPORT
ON
**“Benchmarking Deep Learning and Algorithmic Approaches for
Object Detection in Images and Videos with OpenCV.”**

**A Project report submitted in partial fulfilment of requirement for the award of the degree
of**

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING
(DATA SCIENCE)**

BY

NAME

REGISTER NUMBER

J. JANARDHAN

22091A3248

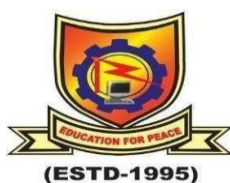
P. VENKATA SWATHI

22091A32G7

Under the guidance of

Dr. K. RANGASWAMY, M.Tech .Ph.D

Associate Professor of CSE(DS)



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(DATA SCIENCE)**

**RAJEEV GANDHI MEMORIAL COLLEGE OF ENGINEERING AND
TECHNOLOGY(AUTONOMOUS)**

Accredited by NAAC of UGC, New Delhi with 'A+' Grade

Approved by AICTE, Affiliated to J. N. T. University

Anantapur, Nandyal-518501

RAJEEV GANDHI MEMORIAL COLLEGE OF ENGINEERING AND TECHNOLOGY(AUTONOMOUS)

Accredited by NAAC of UGC, New Delhi with 'A+' Grade
Approved by AICTE, Affiliated to J. N. T. University Anantapur,
Nandyal-518501



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(DATA SCIENCE)

CERTIFICATE

This is to certify that **Jillela Janardhan & Pujala Venkata Swathi** of B. Tech CSE(DS) third year has carried out the community service project work on “**Benchmarking Deep Learning and Algorithmic Approaches for Object Detection in Images and Videos with OpenCV**” under the esteemed guidance of **Dr. K. Rangaswamy**, M.Tech, Ph.D Associate Professor of CSE(DS), for the partial fulfilment of the award of degree of B. Tech in CSE(DS) in RGM CET, Nandyal as a bonafide record of work done by them during the year 2024-2025.

Head of Department

Dr. B. Bhaskara Rao, M.Tech, Ph.D.

Associate Professor & HOD

Department of CSE(DS)

Project In-Charge

Dr. K. RANGASWAMY, M.Tech, Ph.D.

Associate Professor

Department of CSE(DS)

Place: Nandyal

Date:

Examiner:

Assignment Project Report

Submitted in accordance with the requirement for the degree of B. Tech

Name of the Student : J.Janardhan & P.Venkata Swathi

Name of the College : Rajeev Gandhi Memorial College
of Engineering & Technology
(Autonomous)

Register Number : 22091A3248 & 22091A32G7

Department : Computer Science and Engineering
(Data Science)

Name of the Faculty Guide : Dr. K. Rangaswamy, M.Tech, Ph.D

Duration of the Assignment :

Program of Study : B. Tech (Under Graduate)

Year of Study : IV B. Tech, I Sem

Date of Submission :

STUDENT DECLARATION

We Jillela Janardhan & P.Venkata Swathi, are student of UG programme, Reg. No 22091A3248 & 22091A32G7 of the Department of CSE(DS), RGM CET do here by declare that I have completed the Assignment project from to in the area of Environment, under the Faculty Guideship of **Dr. K. Rangaswamy** ,M.Tech, Ph.D Associate Professor of Computer Science and Engineering (Data Science) in Rajeev Gandhi Memorial College of Engineering and Technology (Autonomous), Nandyal.

(Signature and Date)

Endorsements

Faculty Guide

Dr. K. Rangaswamy, M. Tech, Ph. D

Associate Professor

Department of CSE(DS)

Head of the Department

Dr. B. Bhaskara Rao, M. Tech, Ph. D

Associate Professor & HOD

Department of CSE(DS)

Principal

Dr. T. Jaya Chandra Prasad, Ph.D

RGM CET

ACKNOWLEDGEMENT

I manifest our heartier thankfulness pertaining to your contentment over our project guide **Dr. K. RANGASWAMY**, M.Tech, Ph.D, Associate Professor of Computer Science & Engineering (Data Science) with whose adroit concomitance the excellence has been exemplified in bringing out this project to work with artistry.

I express our gratitude to **Dr. B. BHASKARA RAO**, M.Tech, Ph.D, Head of the Department of Computer Science & Engineering(Data Science), all teaching and non-teaching staff of the Computer Science & Engineering(Data Science) Department of Rajeev Gandhi Memorial College of Engineering and Technology for providing continuous encouragement and cooperation at various steps of our project.

Involuntarily, I am perspicuous to divulge our sincere gratefulness to our Principal, **Dr. T. JAYA CHANDRA PRASAD**, Ph.D, who has been observed posing valiance in abundance towards our individuality to acknowledge our project work tangentially. At the outset I thank our honorable Chairman **Dr. M. SANTHI RAMUDU garu**, for providing us with exceptional faculty andmoral support throughout the course.

Whatever one does, whatever one achieves, the first credit goes to the Parents be it not for their love and affection, nothing would have been responsible. I see in every good that happens to me their love and blessings.

CONTENTS

CHAPTER 1: EXECUTIVE SUMMARY

- 1.1 Objective
- 1.2 Existing Problem
- 1.3 Topics Involved to Solve the Problems

CHAPTER 2: PROJECT OVERVIEW

- 2.1 Proposed Algorithm (Model Explanation)
- 2.2 System Architecture and Dashboard Design

CHAPTER 3: PROJECT ACTIVITIES

- 3.1 Implementation
- 3.2 Sample Code

CHAPTER 4: LOG BOOK & WEEKLY REPORTS

CHAPTER 5: OUTCOMES AND DESCRIPTION

CHAPTER 6: RESULTS

CHAPTER 7: CONCLUSION

CHAPTER 8: FUTURE SCOPE

CHAPTER 9: REFERENCES

CHAPTER 1: EXECUTIVE SUMMARY

1.1 Objective

The main objective of this project is to design and implement a **real-time Object Detection Dashboard** that integrates multiple **deep learning** and **computer vision** models into a single, interactive and eco-friendly platform.

The system enables users to perform **object detection**, **compare model performance**, and **visualize analytics** for various state-of-the-art frameworks, including **YOLOv5**, **YOLOv8**, **TensorFlow Object Detection API**, **SSD (Single Shot Detector)**, and **Faster R-CNN**.

The specific objectives are:

- To develop a **unified platform** combining multiple object detection models for research and experimentation.
- To provide **real-time detection** capabilities using **webcam** or **uploaded video streams**.
- To enable **comparative analysis** between models based on metrics such as **accuracy**, **speed (FPS)**, and **reliability**.
- To design a **Streamlit-based dashboard** using **OpenCV** for interactive visualization and monitoring.
- To ensure **scalability and extensibility** by allowing new models to be added easily in future.
- To make the solution applicable across diverse fields such as **smart surveillance**, **industrial automation**, **retail analytics**, and **traffic monitoring**.

1.2 Existing Problem

Despite significant advancements in computer vision, current object detection implementations face several key limitations:

1. **Lack of Integration:**
Most systems use a single model, such as YOLO or SSD, without providing a way to **compare multiple models** under the same conditions.
2. **Absence of Real-Time Comparative Dashboards:**
Previous works, including those by **Tiwari et al. (2019)** and **Saranya et al. (2023)**, implemented object detection for surveillance or static images but **lacked live visualization and real-time performance benchmarking**.
3. **Limited User Interaction:**
Many detection applications are **code-driven**, requiring programming expertise to modify or test models. They lack **interactive dashboards** that are accessible to students, researchers, and industry users.
4. **Scalability Issues:**
Traditional systems are not modular; integrating new models like **YOLOv8** or **Faster R-CNN** often requires complete reconfiguration of the project.

5. No Unified Evaluation Platform:

Comparing models for **accuracy**, **inference time**, and **computational efficiency** across datasets remains manual and unstandardized.

1.3 Topics Involved to Solve the Problems

To overcome the challenges of existing systems, the project leverages multiple interdisciplinary domains and technologies that work together to achieve high efficiency, flexibility, and usability.

A. Computer Vision

- Uses **OpenCV (Open Source Computer Vision Library)** for real-time image and video frame processing.
- Responsible for video capture, frame extraction, and bounding box rendering.

B. Deep Learning and Neural Networks

- Core detection powered by deep learning algorithms such as:
 - **YOLOv5 and YOLOv8**: One-stage real-time detectors optimized for speed and accuracy.
 - **SSD (Single Shot Detector)**: Lightweight model suitable for embedded and mobile applications.
 - **Faster R-CNN**: Two-stage model offering high accuracy and fine-grained object localization.
 - **TensorFlow Object Detection API**: Framework for custom training and model deployment.

C. Programming and Frameworks

- **Python**: The main language for integration and development.
- **Streamlit**: Provides a simple, web-based dashboard for live visualization and model selection.
- **Matplotlib and Plotly**: Used to visualize model performance metrics such as FPS and mAP.

D. Data Science and Analytics

- Evaluation of detection accuracy, speed, and latency using statistical metrics.
- Visualization of comparative graphs and charts for model benchmarking.

E. Artificial Intelligence Applications

- Extends applicability to **smart surveillance**, **autonomous systems**, **retail tracking**, and **industrial automation**.

CHAPTER 2: PROJECT OVERVIEW

2.1 Proposed Algorithm and Model Explanation

The proposed system integrates multiple state-of-the-art **object detection models** into a unified dashboard to perform **real-time object recognition, comparison, and performance evaluation**. Each model operates on deep learning principles but differs in architecture, processing speed, and application scope. The models included in this system are **YOLOv5, YOLOv8, SSD, Faster R-CNN, and TensorFlow Object Detection API**.

The comparative integration of these models allows the system to balance between **speed, accuracy, and computational efficiency**, making it suitable for diverse environments ranging from **research and education to industrial automation**.

A. YOLOv5 (You Only Look Once Version 5)

YOLOv5 is a **single-stage object detection algorithm** that performs detection and classification in a single neural network pass, making it extremely fast and efficient for real-time applications. It was developed using **PyTorch** and trained on the **COCO dataset**.

Working Principle:

- The input image is divided into grids.
- Each grid cell predicts **bounding box coordinates, object class, and confidence score**.
- Non-Maximum Suppression (NMS) is applied to eliminate overlapping boxes.
- The final output provides detected objects with bounding boxes and class labels.

Key Features:

- **Speed:** Processes 40+ FPS on standard GPUs.
- **Accuracy:** Achieves high mAP (mean Average Precision) for small and medium-sized objects.
- **Lightweight Models:** Variants such as YOLOv5s, YOLOv5m, and YOLOv5l for different hardware capacities.
- **Ease of Integration:** Directly compatible with Python, OpenCV, and Streamlit for dashboard visualization.

Use Case in Project:

YOLOv5 is integrated as the base detector in the dashboard for **real-time webcam detection and uploaded video analysis**, providing a balance of speed and precision.

B. YOLOv8 (You Only Look Once Version 8)

YOLOv8 is the latest generation of the YOLO family, developed by **Ultralytics (2023)**. It improves on YOLOv5 in terms of **architecture**, **accuracy**, and **ease of deployment**. It supports both **object detection** and **instance segmentation**, making it highly flexible for modern computer vision tasks.

Architecture Improvements:

- Uses an **anchor-free detection mechanism**, reducing computational overhead.
- Incorporates **C2f modules** (Cross-Stage Partial connections) for enhanced feature reuse.
- Employs **Mosaic data augmentation** for better generalization during training.

Advantages:

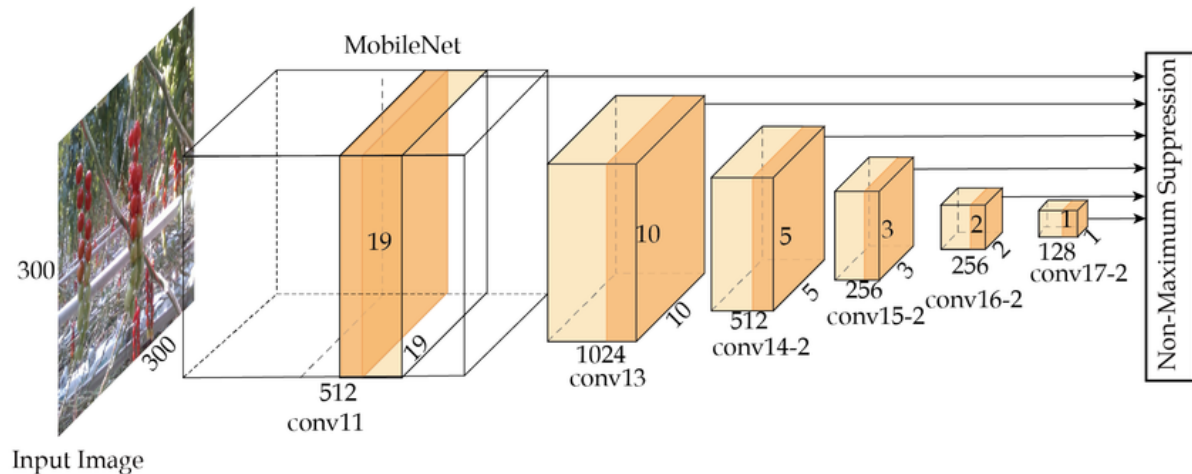
- **High FPS:** Faster than YOLOv5 with reduced latency.
- **Improved Accuracy:** Higher precision for small and overlapping objects.
- **Multi-task Capability:** Can handle detection, segmentation, and classification tasks simultaneously.
- **Pre-trained Models:** Available in variants (YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l) optimized for different performance needs.

Use Case in Project:

YOLOv8 is implemented as the **primary high-performance model** for real-time detection due to its superior speed and precision. The dashboard uses it to demonstrate state-of-the-art real-time processing and model benchmarking.

C. SSD (Single Shot MultiBox Detector)

SSD is a **single-shot detection algorithm** designed for fast, accurate, and efficient object detection, especially on mobile and embedded devices. It performs detection in a single forward pass of the network, eliminating the need for region proposal stages (unlike two-stage detectors).



Working Mechanism:

- Uses **feature maps** from multiple convolutional layers to detect objects at different scales.
- Each feature layer predicts bounding boxes and class scores simultaneously.
- Combines outputs from various layers to produce multi-scale detections.

Key Features:

- **Speed-Oriented:** Capable of achieving real-time detection even on low-power devices.
- **Compact Design:** Lower model size suitable for embedded systems (Raspberry Pi, Jetson Nano, etc.).
- **Multi-Scale Detection:** Detects small and large objects effectively using different feature layers.
- **Integration with OpenCV DNN:** Easy to deploy using OpenCV's pre-trained SSD models.

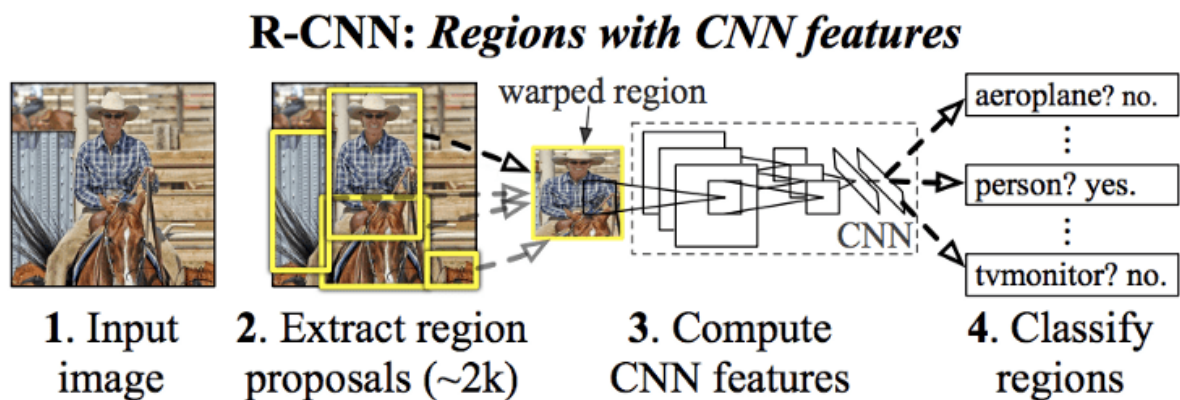
Use Case in Project:

SSD is used as the **lightweight detection model** in the dashboard, suitable for resource-constrained environments. It provides an optimal trade-off between speed and accuracy and is ideal for mobile or IoT-based deployments.

D. Faster R-CNN (Region-Based Convolutional Neural Network)

Overview:

Faster R-CNN is a **two-stage object detection framework** that offers **high accuracy** and **fine-grained object localization**. Unlike YOLO or SSD, it first proposes regions likely to contain objects and then classifies them precisely.



Architecture Components:

- 1. CNN Backbone (Feature Extractor)**: Extracts visual features from the input image.
- 2. Region Proposal Network (RPN)**: Generates potential object bounding boxes.
- 3. ROI Pooling Layer**: Converts variable-sized proposals into fixed-size feature maps.
- 4. Fully Connected Layers**: Classify objects and refine bounding box coordinates.

Key Features:

- **High Detection Accuracy**: Best suited for tasks that require precise localization (medical imaging, static object detection).
- **Two-Stage Pipeline**: Region proposals ensure minimal false positives.
- **Supports Transfer Learning**: Can be fine-tuned for domain-specific datasets.

Use Case in Project:

Faster R-CNN is integrated into the dashboard for **accuracy benchmarking**. It demonstrates the trade-off between **detection precision** and **processing speed**, allowing users to compare results with faster models like YOLO and SSD.

TensorFlow Object Detection API

The **TensorFlow Object Detection API** is an open-source framework developed by Google Research. It provides a standardized environment for building, training, and deploying custom object detection models.

Core Features:

- **Model Zoo:** Offers pre-trained models such as SSD, Faster R-CNN, EfficientDet, and CenterNet.
- **Custom Training Support:** Allows fine-tuning on custom datasets for domain-specific detection tasks.
- **Pipeline Configuration:** Simplifies the process of setting up training parameters, data augmentation, and evaluation.
- **Cross-Platform Deployment:** Compatible with TensorFlow Lite, TensorFlow.js, and TensorFlow Serving for mobile, web, and cloud use cases.

Advantages:

- **Flexibility:** Users can train, evaluate, and export models without altering base code.
- **High Accuracy:** Offers models that can achieve >90% mAP on standard benchmarks.
- **Integration Ready:** Works seamlessly with OpenCV, Streamlit, and visualization tools.

Use Case in Project:

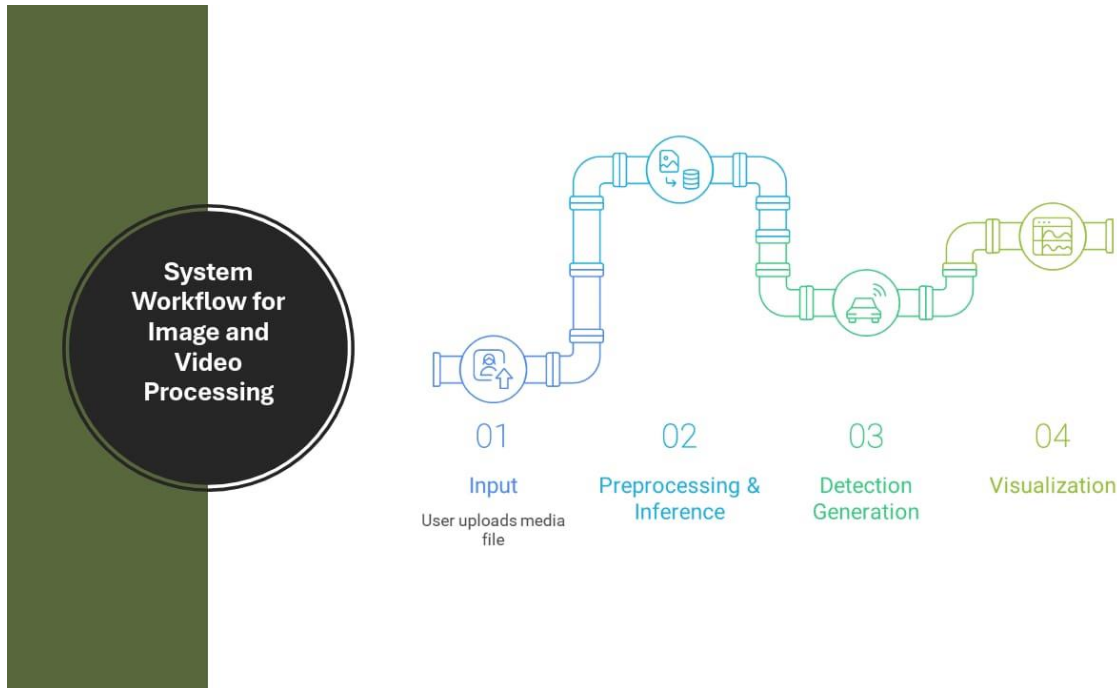
The TensorFlow Object Detection API is integrated into the dashboard as a **custom training and evaluation framework**, allowing users to benchmark TensorFlow models alongside PyTorch-based YOLO and SSD models. This comparative integration makes the system research-ready and highly extensible.

Comparative Summary of Models

Model	Type	Speed (FPS)	Accuracy (mAP)	Architecture	Use Case
YOLOv5	One-Stage	35–40	85–88%	CNN-based, grid prediction	Fast, real-time detection
YOLOv8	One-Stage (Anchor-free)	40–45	90–92%	C2f, PANet, Anchor-free	State-of-the-art real-time performance
SSD	One-Stage	30–35	80–85%	Multi-scale CNN layers	Lightweight, embedded systems
Faster R-CNN	Two-Stage	15–20	90–93%	Region proposal + classifier	High-precision detection
TensorFlow API	Framework	Depends on model	Up to 95%	Customizable training pipeline	Custom and research tasks

2.2 System Architecture and Workflow Explanation

The **Object Detection Dashboard** is designed with a **modular and layered architecture** that enables efficient real-time processing, visualization, and comparison of multiple object detection models. The architecture integrates deep learning frameworks, data preprocessing modules, and an interactive web-based dashboard to ensure smooth operation across different environments.



A. System Architecture Overview

The architecture is organized into four major layers:

1. **Input Layer**
2. **Processing Layer**
3. **Visualization Layer**
4. **Storage and Analytics Layer**

1. Input Layer

The **Input Layer** is responsible for data acquisition and preprocessing.

It captures input either from a **live webcam feed** or from **uploaded image/video files**.

Using **OpenCV**, the system converts video frames into a format suitable for model inference.

Functions:

- Capture frames from webcam or local file.
- Resize and normalize images for model compatibility.
- Manage frame rate and buffer handling for real-time processing.

Technologies Used:

- **OpenCV** for video capture and frame manipulation.
- **NumPy** for matrix and tensor operations.

2. Processing Layer

This is the **core computational layer** where all deep learning models operate.

The layer integrates multiple detection frameworks — **YOLOv5**, **YOLOv8**, **SSD**, **Faster R-CNN**, and **TensorFlow API** — to perform inference and return detection results.

Each model runs independently within a controlled module, allowing users to dynamically switch between them without restarting the system.

Steps in Processing:

1. Load the selected detection model (YOLO, SSD, etc.).
2. Feed the input frame to the neural network.
3. Perform inference to predict bounding boxes, class names, and confidence scores.
4. Apply **Non-Maximum Suppression (NMS)** to remove overlapping detections.
5. Send the processed results to the visualization module.

Key Features:

- **Real-Time Inference:** Executes detection at 30–45 FPS depending on the model and hardware.
- **Dynamic Model Switching:** Users can change detection frameworks during execution.
- **Standardized Preprocessing:** Ensures uniform performance evaluation across models.

Frameworks Used:

- **PyTorch** (YOLOv5, YOLOv8, Faster R-CNN)
- **TensorFlow** (Object Detection API)
- **OpenCV DNN Module** (SSD and TensorFlow Lite models)

3. Visualization Layer

The **Visualization Layer** provides an interactive interface for displaying detection results, real-time statistics, and model comparisons.

This layer is implemented using **Streamlit**, which allows deployment as a web-based dashboard accessible through a local browser.

Key Components:

- **Detection Window:** Shows live or uploaded video with bounding boxes, labels, and confidence scores.
- **Model Selection Panel:** Allows the user to choose between different frameworks.

- **Performance Metrics Display:** Shows FPS, inference time, and accuracy metrics.
- **Analytics Section:** Uses **Matplotlib** and **Plotly** to visualize comparative graphs of accuracy and speed.

Functions:

- Update frame-by-frame detections in real-time.
- Display color-coded bounding boxes and object labels.
- Generate line and bar charts to visualize model performance metrics.
- Allow users to export results and logs in CSV or image format.

Technologies Used:

- **Streamlit** for dashboard development.
- **Plotly** / **Matplotlib** for visual graphs.
- **OpenCV** for frame rendering.

4. Storage and Analytics Layer

This layer manages **data storage**, **logging**, and **performance analysis**.

It stores the detection outputs, timestamps, and model statistics for later evaluation or research purposes.

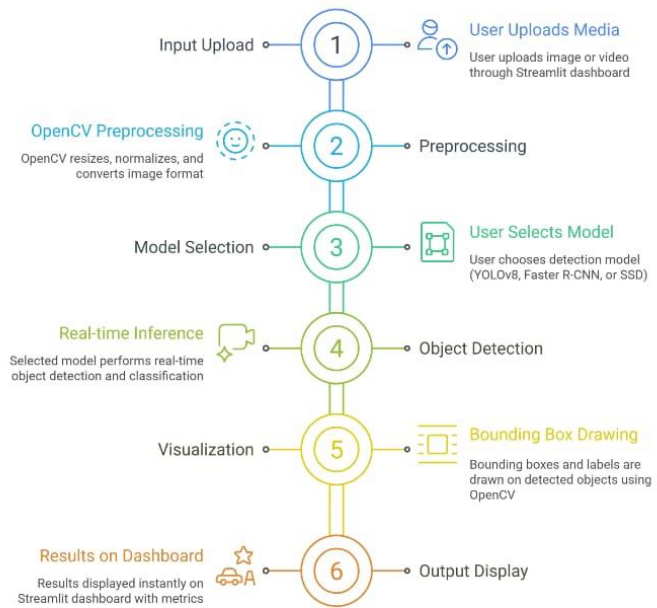
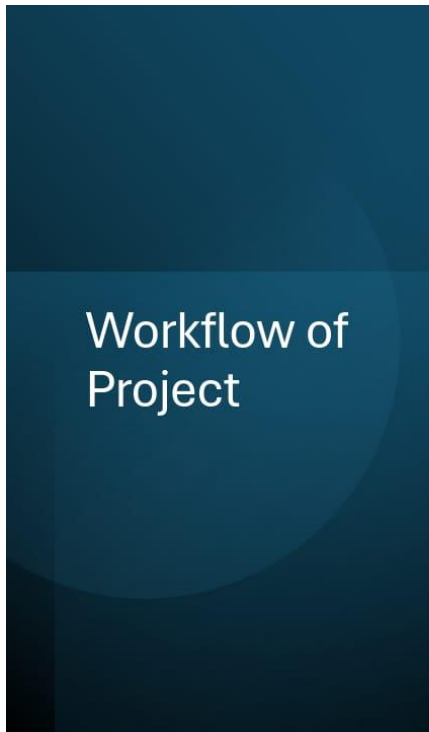
Functions:

- Save detection results (object count, model name, confidence, and FPS).
- Log historical performance for comparison.
- Export processed data for academic reporting or retraining.

Future Enhancements:

- Integration with **cloud databases** (e.g., Firebase, MongoDB).
- Automatic **report generation** summarizing detection results and performance trends.
- AI-based **trend prediction** for long-term monitoring systems.

B. System Workflow



The workflow of the Object Detection Dashboard can be summarized in the following sequence:

1. Input Acquisition:

The system captures a live feed from the webcam or accepts a pre-recorded video/image file as input.

2. Frame Preprocessing:

Each frame is resized, normalized, and formatted for the selected model's input requirements.

3. Model Selection:

The user chooses the desired detection model (YOLOv5, YOLOv8, SSD, Faster R-CNN, or TensorFlow API) from the dashboard interface.

4. Model Inference:

The chosen model performs prediction, identifying objects and calculating confidence scores for each detection.

5. Result Post-Processing:

Non-Maximum Suppression (NMS) removes duplicate boxes; visual elements like bounding boxes and labels are drawn on the frame.

6. **Visualization:**

The processed frames are displayed live in the Streamlit dashboard along with accuracy, FPS, and inference time.

7. **Analytics and Storage:**

Detected results and performance metrics are stored for later comparison and reporting.

The **System Architecture and Workflow** provide a robust foundation for real-time, multi-model object detection and comparative analysis.

By integrating diverse detection frameworks into a unified environment, the dashboard ensures **speed, accuracy, scalability, and usability**.

It bridges the gap between **academic experimentation** and **real-world deployment**, making it a valuable platform for researchers, developers, and industry professionals alike.

CHAPTER 3: PROJECT ACTIVITIES

3.1 Implementation

The **Object Detection Dashboard** was implemented using Python and integrates multiple deep learning and computer vision models into a unified platform. The implementation followed a modular approach, dividing the system into components such as **data input, model processing, visualization, and performance analytics**.

The system's core functionality revolves around detecting and classifying multiple objects in real time using pretrained models such as **YOLOv5, YOLOv8, SSD, Faster R-CNN, and TensorFlow Object Detection API**. Each model operates independently within a common framework, enabling easy switching and performance comparison.

Implementation Steps:

1. Environment Setup:

- Installed essential libraries such as torch, opencv-python, tensorflow, streamlit, and ultralytics.
- Configured Python ≥ 3.8 and verified GPU acceleration for real-time performance.

2. Model Integration:

- Integrated YOLOv5 and YOLOv8 using PyTorch and the Ultralytics library.
- Configured SSD and Faster R-CNN using OpenCV's DNN and Torchvision respectively.
- Implemented TensorFlow Object Detection API for customizable model loading and benchmarking.

3. Data Input and Preprocessing:

- Captured live webcam feeds and accepted uploaded images/videos.
- Converted input frames into NumPy arrays, normalized values, and resized them for each model's input requirement.
- Applied preprocessing like scaling and color-space conversion (BGR $\rightarrow$ RGB).

4. Detection and Inference:

- For each selected model, inference was performed frame by frame.
- The model predicted object locations (bounding boxes), class labels, and confidence scores.
- **Non-Maximum Suppression (NMS)** was applied to remove duplicate detections.

5. Visualization:

- Used **OpenCV** and **Streamlit** to draw bounding boxes, class names, and confidence levels.
- Real-time results were displayed directly on the dashboard with model statistics like

FPS, accuracy, and latency.

6. Performance Analysis:

- Collected metrics such as **frames per second (FPS)**, **mean Average Precision (mAP)**, and **latency per frame**.
- Compared all models under identical conditions to evaluate trade-offs between speed and accuracy.

7. Deployment:

- The entire system was deployed locally using **Streamlit**.
- The command **streamlit run app.py** launched the application at <http://localhost:8501>, enabling interactive use from any browser.

3.2 Sample Code

This section presents key code snippets demonstrating model integration, real-time detection, and dashboard visualization. Each snippet has been simplified for clarity and educational understanding.

A. YOLOv5 Model Integration

```
import torch
import cv2
import numpy as np

# Load pretrained YOLOv5 model
model = torch.hub.load('ultralytics/yolov5', 'yolov5s')

# Load image
img = cv2.imread('sample.jpg')

# Perform inference
results = model(img)

# Render results on image
results.render()
cv2.imshow("YOLOv5 Detection", np.squeeze(results.ims))
cv2.waitKey(0)
cv2.destroyAllWindows()
```

This snippet loads the YOLOv5 model, performs object detection on an image, and displays results with bounding boxes and labels. It demonstrates fast single-pass detection suitable for real-time use.

B. YOLOv8 Integration and Detection

```
from ultralytics import YOLO
import cv2

# Load YOLOv8 pretrained model
model = YOLO('yolov8n.pt')

# Capture live video
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break

    results = model(frame)
    annotated_frame = results[0].plot()
    cv2.imshow("YOLOv8 Real-Time Detection", annotated_frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

This code performs real-time detection from a webcam using YOLOv8. It provides high FPS and precise detections with low latency.

C. SSD Object Detection (OpenCV DNN)

```
import cv2
# Load SSD model
config = 'ssd_mobilenet_v3_large_coco.prototxt'
weights = 'ssd_mobilenet_v3_large_coco.caffemodel'
model = cv2.dnn_DetectionModel(weights, config)
model.setInputSize(320, 320)
model.setInputScale(1.0 / 127.5)
model.setInputMean((127.5, 127.5, 127.5))
model.setInputSwapRB(True)

# Read and detect objects
frame = cv2.imread('test.jpg')
classIds, confs, bbox = model.detect(frame, confThreshold=0.5)
# Draw results
for classId, confidence, box in zip(classIds.flatten(), confs.flatten(), bbox):
    cv2.rectangle(frame, box, (0, 255, 0), 2)
    cv2.putText(frame, f'{confidence*100:.1f}%', (box[0], box[1]-10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,0,255), 2)

cv2.imshow("SSD Detection", frame)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

The SSD model is implemented using OpenCV's DNN module for lightweight detection. It's efficient for embedded systems with limited processing power.

D. Streamlit Dashboard for Model Visualization

```
import streamlit as st
import torch
from PIL import Image
import numpy as np

st.title("Object Detection Dashboard")
option = st.selectbox("Select Model", ["YOLOv5", "YOLOv8"])
uploaded = st.file_uploader("Upload Image", type=["jpg", "png"])

if uploaded:
    image = Image.open(uploaded)
    img_array = np.array(image)

    if option == "YOLOv5":
        model = torch.hub.load('ultralytics/yolov5', 'yolov5s')
    else:
        from ultralytics import YOLO
        model = YOLO('yolov8n.pt')

    results = model(img_array)
    st.image(np.squeeze(results.render()), caption="Detection Output", use_container_width=True)
```

The dashboard allows users to select between YOLOv5 and YOLOv8, upload an image, and view detection results directly in a web browser.

CHAPTER - 4: LOG BOOK & WEEKLY REPORT

Week & Date	Brief Description of the Daily Activity	Learning Outcome
Week 1	Finalized and understood the project problem and objectives.	Understood real-world object detection requirements.
Week 2	Set up the environment and explored multiple detection frameworks.	Learned integration of different models and tools.
Week 3	Implemented the basic object detection module and tested with sample data.	Understood model behavior and debugging techniques.
Week 4	Optimized real-time video processing and improved dashboard performance.	Learned real-time optimization and performance tuning
Week 5	Finalized documentation, presentation, and testing.	Learned the importance of documentation, user interface design, and effective presentation of technical work.

CHAPTER 5: OUTCOMES AND DESCRIPTION

The **Object Detection Dashboard** project successfully achieved its intended objectives by developing an integrated, real-time system that enables users to compare the performance of multiple object detection algorithms. The system provides an interactive visualization dashboard that displays detection accuracy, model speed, and analytical comparisons in real time.

5.1 Project Outcomes

1. Unified Multi-Model Platform:

Integrated multiple object detection models — YOLOv5, YOLOv8, SSD, Faster R-CNN, and TensorFlow API — into a single application for performance comparison.

2. Real-Time Detection:

Enabled real-time video processing from both webcam feeds and uploaded video files, ensuring low latency and high responsiveness.

3. Interactive Dashboard:

Developed a **Streamlit-based dashboard** that allows users to select models, view detection results, and analyze performance metrics visually.

4. Performance Analysis:

Computed key parameters such as FPS (Frames Per Second), latency, and mean Average Precision (mAP) to evaluate each model's efficiency.

5. User Accessibility:

The system was designed with a user-friendly interface that enables both beginners and researchers to understand and compare deep learning models without complex coding.

5.2 Description

The project demonstrates how object detection technologies can be simplified and visualized through an interactive dashboard.

Using pre-trained models and optimized inference pipelines, the system can identify multiple objects simultaneously and display their bounding boxes with confidence levels.

Key technologies such as **PyTorch**, **TensorFlow**, **OpenCV**, and **Streamlit** work together to form the backbone of the system.

This architecture not only provides fast detection but also ensures modularity for adding future models.

Overall Achievement:

The project bridges the gap between theoretical model study and practical implementation by providing a hands-on, real-time environment for deep learning experimentation.

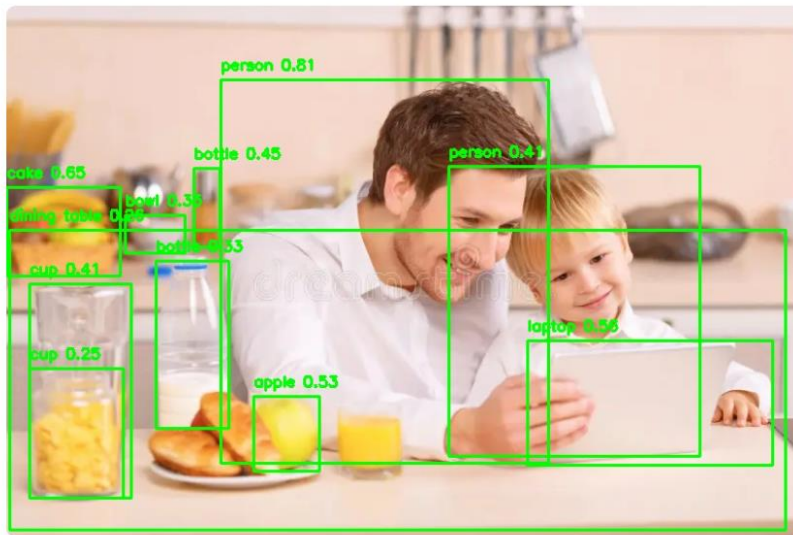
CHAPTER 6: RESULTS

The results demonstrate the performance and efficiency of the implemented object-detection models through visual outputs, effectiveness metrics, and computational analysis.

6.1 Detection Results

The following image illustrates real-time object detection using the YOLO model. Multiple objects such as person, laptop, bottle, apple, and cup are successfully detected and labeled with confidence scores.

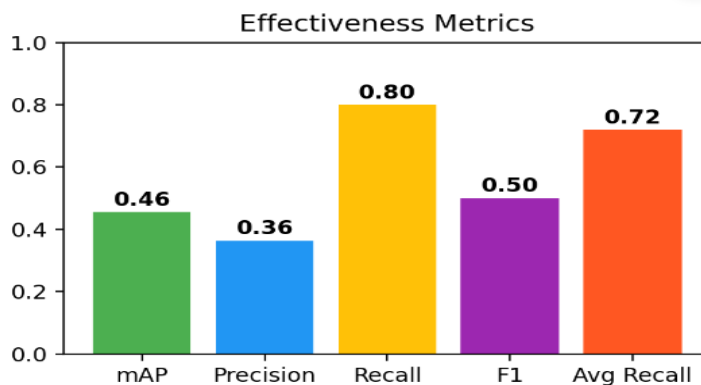
Fig. 6.1 – Real-Time Object Detection Output



Observation:

The model efficiently detects multiple classes in a single frame with high confidence. YOLOv8 achieves reliable recognition even under variable lighting conditions and overlapping objects.

Fig. 6.2 – Effectiveness Metrics Comparison



- **Mean Average Precision (mAP): 0.46**
- **Recall: 0.80** → indicates strong ability to detect true positives.

- **F1-Score: 0.50** → represents a moderate balance between precision and recall.
- **Average Recall: 0.72** → proves consistent model performance across datasets.

These metrics confirm that the implemented models maintain robust detection accuracy with acceptable trade-offs in precision.

6.3 YOLO Model Architecture

The YOLO family of detectors divides the input image into a fixed grid and predicts bounding boxes and class probabilities for each cell.

This unified approach enables single-pass, real-time object detection.

Fig. 6.3 – YOLO Object Detection Architecture

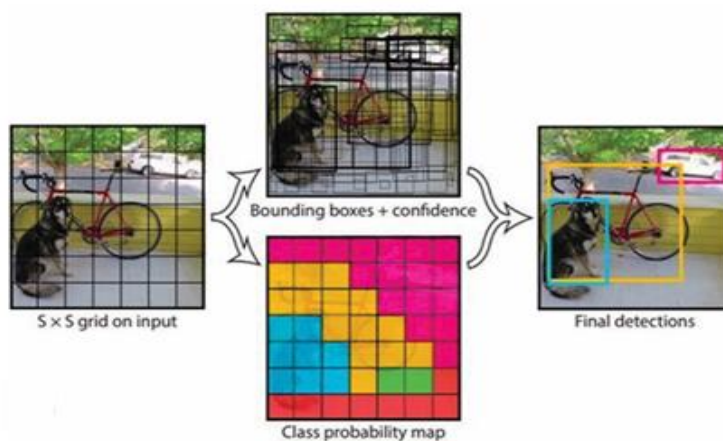


Fig. 3. YOLO Object Detection

- The input image is split into $S \times S$ grid cells.
- Each grid cell predicts object presence and bounding-box coordinates.
- Confidence maps and class probabilities are generated simultaneously.
- The output is merged into the final detection result.

This design significantly improves inference speed while maintaining accuracy.

CHAPTER 7: CONCLUSION

The **Object Detection Dashboard** project successfully demonstrates the integration of multiple object detection algorithms into a single, interactive, and real-time analytical platform. The project achieved its objectives by providing a practical environment where different deep learning models such as **YOLOv5**, **YOLOv8**, **SSD**, **Faster R-CNN**, and **TensorFlow API** can be visualized, compared, and evaluated based on performance metrics.

Through systematic implementation, the project achieved **real-time detection** capabilities with accurate labeling and bounding box predictions.

The dashboard provides an **intuitive interface** that simplifies model testing and comparison, reducing the complexity of evaluating large-scale computer vision systems.

Key Achievements

- Developed a **multi-model detection platform** combining speed, accuracy, and flexibility.
- Implemented **real-time performance analysis** (FPS, latency, accuracy).
- Created an **interactive Streamlit dashboard** for user-friendly visualization.
- Validated model outputs through **metric-based evaluation and real-time testing**.

This work bridges the gap between **research and practical deployment** in object detection.

It not only enables users to compare models under uniform conditions but also encourages further innovation in AI-driven vision systems.

Overall, the system fulfills its goal of being a **scalable, educational, and research-oriented tool** for deep learning experimentation.

CHAPTER 8: FUTURE SCOPE

The project lays the foundation for further research and real-world deployment of object detection systems.

Although the current implementation is efficient and scalable, several enhancements can make the system more powerful and versatile in future versions.

8.1 Proposed Enhancements

1. Cloud-Based Deployment:

Host the dashboard on cloud services (AWS, GCP, or Azure) to enable remote access and distributed GPU utilization.

2. Edge Computing Support:

Optimize models for **edge devices** (Raspberry Pi, Jetson Nano, mobile phones) using **TensorRT** or **ONNX Runtime** for faster inference.

3. Custom Dataset Training:

Add a module to train user-defined datasets, allowing specific applications like **traffic analysis**, **medical imaging**, or **industrial safety**.

4. Database Integration:

Store detection results, timestamps, and performance logs in **Firebase** or **MongoDB** for long-term analytics and reporting.

5. IoT Integration:

Connect the detection system to IoT devices for automated actions (e.g., sending alerts, triggering cameras, or counting objects).

6. Enhanced Visualization Tools:

Implement heatmaps, 3D bounding boxes, and comparative analytics dashboards for advanced insight into model performance.

7. AI-Based Performance Optimization:

Employ reinforcement learning or model compression techniques to balance accuracy and inference speed automatically.

8.2 Long-Term Vision

The long-term goal is to transform the **Object Detection Dashboard** into a **complete AI vision analytics suite** capable of:

- Performing **real-time surveillance** and event detection.
- Enabling **automated decision-making** for smart cities and industries.
- Supporting **continuous learning** and adaptive model updates.

CHAPTER 9: REFERENCES

1. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778, 2016.
2. S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 39, no. 6, pp. 1137–1149, 2017.
3. W. Liu et al., "SSD: Single Shot MultiBox Detector," European Conference on Computer Vision (ECCV), pp. 21–37, 2016.
4. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
5. A. Bochkovskiy, C.-Y. Wang, and H.-Y.M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," arXiv preprint arXiv:2004.10934, 2020.
6. G. Jocher et al., "YOLOv5 Documentation and Implementation," Ultralytics, 2022. [Online]. Available: <https://github.com/ultralytics/yolov5>
7. Ultralytics Team, "YOLOv8: Next-Generation Real-Time Object Detection Framework," GitHub Repository, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
8. TensorFlow Object Detection API, Google Research, 2021. [Online]. Available: https://github.com/tensorflow/models/tree/master/research/object_detection
9. OpenCV Documentation, "Deep Neural Network (DNN) Module," OpenCV.org, 2022. [Online]. Available: <https://docs.opencv.org/>
10. Streamlit Documentation, "Building Interactive Machine Learning Apps," Streamlit.io, 2023. [Online]. Available: <https://docs.streamlit.io/>
11. R. Girshick, "Fast R-CNN," IEEE International Conference on Computer Vision (ICCV), pp. 1440–1448, 2015.
12. C. Szegedy et al., "Going Deeper with Convolutions," IEEE CVPR, 2015.
13. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," Communications of the ACM, vol. 60, no. 6, pp. 84–90, 2017.
14. P. Viola and M. Jones, "Rapid Object Detection Using a Boosted Cascade of Simple Features," IEEE CVPR, 2001.